

Pickaxe API review

What the Pickaxe documentation covers, and how it relates to the Grow Coach app (`grow-za`)

This is a read-through of the Pickaxe documentation at `pickaxe.co/v1/documentation` (22 pages, fetched on 3 Oct 2026), compared with the code in `grow-za` at commit `de14da5` . Nothing was run against the live Pickaxe API; where this document says the code "matches" the docs, that means the documented request shape agrees with what the code sends.

Headline findings:

- 1. The app's use of the Pickaxe API is consistent with the documented endpoints and fields.
- 2. Two behaviours the app relies on (memory writes that overwrite, and 404s for unknown users) are *not documented*. They were found by testing, so they could change without notice.
- 3. Pickaxe passes the signed-in user's email to actions as an environment variable. The two Coach Max actions do not use it, and instead trust whatever the model passes, which is a real security gap.
- 4. The docs give a supported route to branding the sign-in email and to removing a hardcoded portal URL.

Contents

- 1. Scope: what was and was not read
 - 2. Pickaxe at a glance
 - 3. The endpoints this app uses
- 4. Docs versus code, point by point
 - 5. Features the app does not use yet
 - 6. Recommendations
 - 7. Open questions to verify

1. Scope: what was and was not read

Depth	Pages
Read in full	Introduction, Memory, User, Prompt Injectors, Environment Variables, Webhooks, Authenticated Embeds, Email Templates, CLI & Coding Agents
Read in part (opening sections, field lists, response shapes)	Access Group, Portal, Action
Endpoint lists only	Workspace, Agent, Completions, Insights, MCP, Deployment. Internal Pickaxe MCP, Document and Page were fetched but not reviewed.

Conclusions about those last pages would be guesses, so none are drawn here.

2. Pickaxe at a glance

Pickaxe is a no-code platform for building and deploying AI tools. The documentation describes a versioned HTTP API under <https://api.pickaxe.co/v1>. Workspace endpoints use `Authorization: Bearer YOUR_API_TOKEN`; deployment completions use the deployment's own API key.

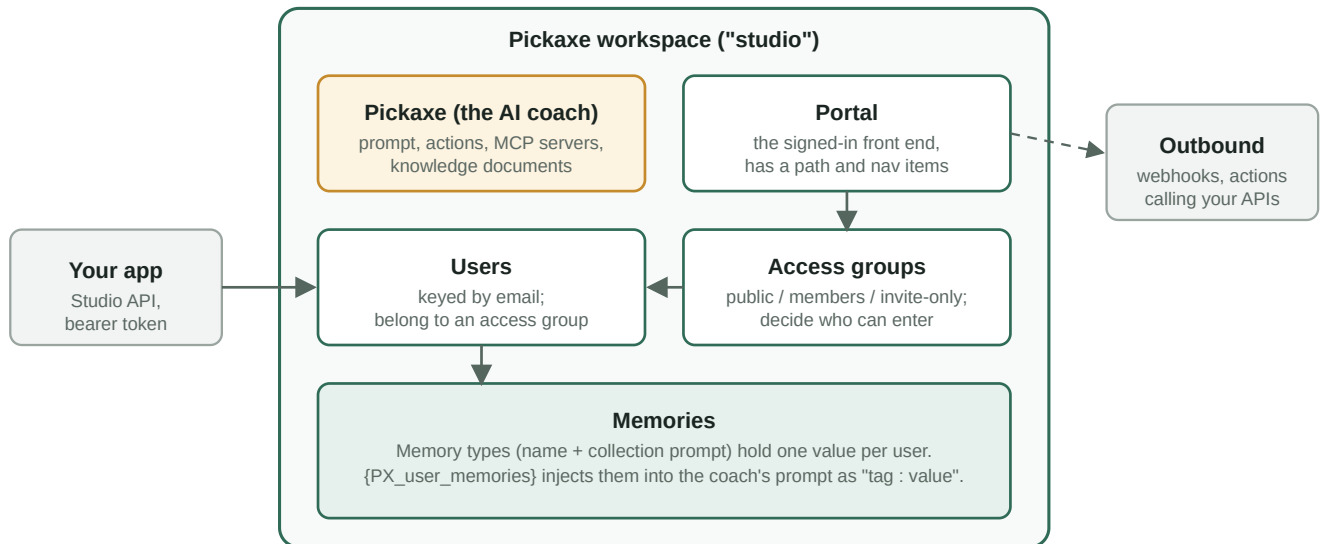


Figure 1. The Pickaxe resource model as the documentation describes it. Access groups gate entry to the portal, users belong to access groups, and memories are per-user values the coach can read.

3. The endpoints this app uses

Endpoint (under <code>/v1/studio</code>)	Documented fields	Used in grow-za
<code>GET /memory/user/<user_id></code>	<code>memoryId</code> (optional filter), <code>skip</code> , <code>take</code> (default 10)	<code>getUserMemory</code> passes ? <code>memoryId=</code> to read the Onboarding Complete flag.
<code>POST /memory/user/create</code>	<code>userId</code> , <code>memoryId</code> , <code>value</code> (all required)	<code>setUserMemory</code> for Name, Has Grow Coach, Manager Name, Manager Email and Onboarding Complete.
<code>PATCH</code> <code>/memory/user/<email>/<memory_id></code>	<code>data.value</code>	Fallback inside <code>setUserMemory</code> , and <code>clear-test-user.sh</code> .
<code>DELETE /memory/user/<email></code>	optional <code>memoryId</code>	Hard-delete mode of <code>clear-test-</code> <code>user.sh</code> .
<code>GET /user/<email></code>	email in the path	<code>ensureStudioUser</code> lookup.
<code>POST /user/create</code>	<code>email</code> (required); <code>name</code> , <code>accessGroupId</code> , <code>isEmailVerified</code> , <code>password</code> , others optional	<code>ensureStudioUser</code> creates the coachee with <code>accessGroupId</code> and <code>name</code> .

POST /action/manifest , POST /action/connect	YAML manifest; pickaxeId , actionId , optional triggerPrompt	Not called by code. The two *.pickaxe-action.yaml files are in this format and can be uploaded this way.
GET /portal/list , GET /access-group/list	no body	Not called by code. Referenced in .env.example as the place to find the portal path and access-group id.

4. Docs versus code, point by point

Topic	Verdict	Detail
Filtered memory read	Matches	The docs list memoryId as an optional filter and take as defaulting to 10. The code's reason for filtering ("the unfiltered list paginates at 10 rows") is exactly what the docs describe.
Memory write shape	Matches	userId , memoryId , value as documented. The PATCH fallback correctly wraps the value in data .
Writes overwrite existing values	Not documented	The docs call the endpoint "create a collected memory specific to a user" and say nothing about overwriting. The README states this was confirmed against the live API. The PATCH fallback means the write path degrades safely if that changes.
404 for unknown users	Not documented	The docs describe no error responses for user or memory lookups. The read path treats a 404 as "nothing stored" and the create path treats it as "must create the user". Both rely on observed behaviour.
User creation	Matches	accessGroupId and name are documented optional fields. The docs mark products as deprecated in favour of accessGroupId , which is what the code uses.
isEmailVerified	Unused option	Defaults to false . The code does not set it. Whether setting it would change the magic-link sign-in is not stated in the docs, so test before relying on it.
Invite behaviour	Partly documented	The docs describe POST /user/invite but not the "already existing members" response the README reports.
Portal list contents	Check	The README says the portal list returns studioId and path . The documented example response shows portalId and path but no studioId . GET /studio/whoami (Workspace page) may be the better source. Verify.
Action manifests	Compatible	The repo's YAML files use the documented manifest format, and the docs' upload and attach endpoints are the way to automate installing them.

5. Features the app does not use yet

5.1 Runtime context passed to actions

When Pickaxe runs an action it injects environment variables. The ones that matter here:

Variable	Contents
PICKAXE_USER_EMAIL	The resolved end-user email, when available.
PICKAXE_USER_IDENTIFIER	The end-user identifier for the run (usually the email).
PICKAXE_USER_MEMORIES	JSON array of formatted memory strings.
PICKAXE_RESPONSE_ID	The current session or conversation id.

The docs also describe per-user secrets (`POST /studio/user/secrets/create`), which become environment variables in actions for that user only.

Why it matters: both Coach Max actions currently take the coachee's email as a model-supplied argument. A model error, or a user who talks the coach into using someone else's address, would make the app email another person.

5.2 Prompt injectors

`{PX_user_memories}` is replaced with visible, active memories, one per line as `tag : value`, or "No memories found". `{PX_user_identifier}` and `{PX_current_time}` are also available. This is how the Name, Manager and Coaching Goals memories reach the coach. The memory PATCH endpoint has an `isEnabled` field, so a disabled memory may not be injected. Check that the memories this app writes are enabled.

5.3 Email templates

`GET` and `PATCH /studio/emails/<type>` manage four templates: `magicLink`, `emailVerification`, `passwordReset` and `invitation`. Templates accept `{{studio.name}}`, `{{studio.image}}` and `{{url}}`. The onboarding wizard's last step tells people to request a magic link, so that email is part of the journey and could carry the same branding as the Resend emails.

5.4 Webhooks

Configured in workspace settings, not through the API, and a Pro-plan feature. Events: `signup`, `payment`, `membership`, `conversation` (after 30 minutes of inactivity). Each payload carries `studioId`, `studioName`, `userEmail`, `giftedProducts` and `event`. This could confirm that someone completed sign-in after onboarding, which the app cannot currently see.

5.5 Authenticated embeds

Visitors already signed in to your own site can be signed in to an embedded Pickaxe with a short-lived ES256-signed token. It applies to *embeds on your own website*. The app currently sends people to the Pickaxe portal, so this would be a design change, not a drop-in replacement for the magic link.

5.6 CLI

The Pickaxe CLI (`@pickaxe/pickaxe-cli`) covers users, memory, access groups, portals, actions and deployments, plus `init` / `pull` / `diff` / `push` to sync a project. It would let the Pickaxe-side setup (actions, memory types, portal) be scripted and reviewed like code.

6. Recommendations

#	Change	Effort	Reason
1	In both actions, read the coachee email from <code>PICKAXE_USER_EMAIL</code> instead of an argument	Small	Removes the "model supplies identity" risk. Confirm the variable is populated in your portal context first.
2	Add a shared secret header, set per action as an environment variable, checked by <code>/api/invite</code> and <code>/api/send-manager-email</code>	Small	Closes the unauthenticated-email problem raised in the project overview.
3	Replace the hardcoded portal URL in <code>send-invite.ts</code> with an environment variable	Small	Use the portal list or <code>NEXT_PUBLIC_PICKAXE_BOT_URL</code> , which already holds the same value.
4	Customise the <code>magicLink</code> email template	Small	Consistent branding at the one email the app does not control.
5	Check that the memories Coach Max reads (Coaching Goals, Goal Progress, Committed Actions, Key Relationships, Patterns Observed) are enabled	Small	The coach's system prompt reads them, and only visible, active memories reach it through <code>{PX_user_memories}</code> . Coach Max sets goals itself in its first conversation, so no separate goal-setting step is needed.
6	Add a small test or startup check for the two undocumented behaviours (overwrite on write, 404 on unknown user)	Medium	They are the assumptions most likely to break silently.
7	Consider the CLI for versioning Pickaxe-side configuration	Medium	Actions, memory types and portal would live in source control.

7. Open questions to verify

- Does `PICKAXE_USER_EMAIL` populate for portal sessions (as opposed to embeds), and for a user created through the API before first sign-in?
- Does `isEmailVerified: true` on user creation change what the magic-link flow does?
- Does a disabled memory (`isEnabled: false`) disappear from `{PX_user_memories}`, and are the memories this app writes enabled by default?
- Where does `studioId` come from: the portal list, `whoami`, or elsewhere?
- What is the maximum `take` value on the list endpoints? The docs give only the default of 10.
- The pages marked "endpoint lists only" in section 1 may contain details that change these conclusions.

Source: pickaxe.co/v1/documentation and its 21 sub-pages, fetched 3 Oct 2026. Code: EmmanuelAzu/grow-za at de14da5. No live API calls were made. Revised 3 Oct 2026: recommendation 5 corrected after reading the Coach Max system prompt.